Database Design Report Document

Author: RWOTOMARA JOSHUA HOWARD
Reg No: 2025/BCS/169/PS

wednesday, 6th May 2026

Revision History

Date	Version	Author	Change
23, April 2026	1.0	Joshua Howard Rwotomara	Initial report written
28, April 2026	1.0	Joshua Howard Rwotomara	Included the ER Diagram in the database Design
6, May 2026	1.0	Joshua Howard Rwotomara	Changed Formatting of the document

Table of Contents

1 Purpose

The purpose of this Database Design Document is to provide a comprehensive description of the design, structure, and implementation of the Inventory Stock Management System database. It serves as a formal reference that translates the system's conceptual and logical data models into a well-defined physical database design using a relational database management system. This document outlines the database architecture, entities, relationships, constraints, and design decisions to ensure clarity, consistency, and accuracy during development and implementation. Additionally, it acts as a communication tool among developers, database administrators, and other stakeholders by clearly defining how data is stored, accessed, and managed within the system. The document also provides a foundation for future maintenance, scalability, and enhancements, ensuring that the database can evolve alongside changing system requirements while maintaining data integrity, performance, and security.

1.1 Document Objectives

The objectives of this Database Design Document are to clearly define and communicate the structure and design of the Inventory Stock Management System database, ensuring a shared understanding among all stakeholders involved in the project. Specifically, this document aims to describe how data is organized, stored, and related within the system, providing a detailed blueprint for database implementation using a relational database management system. It also seeks to guide developers and database administrators in building, testing, and maintaining the database by outlining key components such as tables, relationships, constraints, and design decisions. Furthermore, the document serves as a reference for ensuring data integrity, consistency, and security while supporting efficient data access and transaction processing. Ultimately, it provides a foundation for future system enhancements, scalability, and integration with other applications or user interfaces

1.2 Acronyms and Abbreviations

Acronym / Abbreviation	Meaning
RDMS	Relational Database Management System
DBMS	Database Management System
SQL	Structured Query Language
ERD	Entity Relationship Diagram
EER	Enhanced Entity Relationship

2 Assumptions, Constraints and Dependencies

2.1 Assumptions

The design of the Inventory Stock Management System database is based on several assumptions regarding its usage and environment. It is assumed that the system will be used by authorized personnel such as employees and administrators who interact with the database either directly or through a user interface. The database is expected to operate within a stable environment using MySQL as the relational database management system. It is also assumed that all data entered into the system, including product, supplier, and transaction details, will be accurate and validated before storage. Furthermore, the system assumes a moderate level of transaction volume suitable for small to medium-sized operations. Each transaction is expected to involve at least one product and be associated with relevant entities such as employees, customers, or suppliers. The design also assumes that future enhancements can be added without significantly altering the existing schema.

2.2 Constraints

The database design is subject to several constraints that may affect its implementation and operation. The system is limited to the use of MySQL, which defines the supported data types, indexing methods, and constraints. Hardware limitations such as storage capacity and processing power may impact performance, especially as data volume grows. Time constraints related to project deadlines may also limit the extent of advanced features such as triggers, stored procedures, or optimization techniques. Additionally, the system currently lacks a fully developed user interface, meaning interaction may rely heavily on SQL queries during initial stages. Security implementation may also be basic in the initial version, with more advanced controls planned for future development.

2.3 Dependencies

The functionality of the database depends on several external and internal factors. It relies on the proper installation and configuration of the MySQL server and supporting tools such as MySQL Workbench. The system also depends on accurate input data to ensure meaningful outputs and reports. Future enhancements, such as frontend applications or APIs, will depend on the stability and structure of the existing database schema. Additionally, the database may depend on integration with other software components or systems for extended functionality, such as reporting tools or authentication systems. Reliable system hardware and operating system support are also essential for consistent database performance and availability.

3 System Overview

The Inventory Stock Management System is a database-driven solution designed to efficiently manage and monitor inventory-related operations within an organization. The system provides a centralized platform for storing and managing data associated with products, suppliers, customers, employees, and transactions. Its primary function is to facilitate accurate tracking of stock levels, record purchase and sales activities, and maintain a reliable history of all inventory movements.

The system is built using a relational database model implemented in MySQL, where data is organized into interconnected tables that represent key entities such as Product, Supplier, Customer, Employee, Role, Transaction, and TransactionItem. These entities are linked through defined relationships to ensure data consistency and integrity while supporting efficient data retrieval and manipulation.

Through this structure, the system enables users to perform essential operations such as adding new products, updating stock quantities, recording transactions, and generating reports. It also ensures accountability by associating transactions with specific employees and linking purchases and sales to suppliers and customers respectively. The design supports scalability, allowing the system to accommodate future enhancements such as automated processes, user interfaces, and integration with external applications.

Overall, the system serves as a reliable foundation for inventory management by improving data organization, reducing manual errors, and supporting informed decision-making through accurate and accessible data

3.1 Database Management System Configuration

The Inventory Stock Management System database is implemented using the MySQL Relational Database Management System (RDBMS), which provides a reliable and efficient platform for data storage, retrieval, and management. MySQL was selected due to its robustness, scalability, ease of use, and wide support for structured query operations. The database is designed to run on a standard computing environment with support for SQL-based operations and relational data modeling.

The configuration of the DBMS includes the installation and setup of the MySQL server, which hosts the database, and client tools such as MySQL Workbench for database design, querying, and administration. The system operates using standard SQL for defining schema structures (DDL), manipulating data (DML), and executing queries for reporting and analysis.

The database is expected to run on a local or networked machine with adequate processing power, memory, and storage to support moderate transaction volumes. Default MySQL configurations are assumed, with support for features such as indexing, foreign key constraints, and data validation to ensure integrity and performance. Additionally, the system allows for future configuration enhancements, including optimization, security settings, and backup scheduling, as the application scales or moves into production environments.

Vendor	Product	Version	Restrictions
Oracle Corporation	MySQL	8.x	Limited to MySQL-supported features; performance depends on available hardware resources; advanced features such as triggers and stored procedures may not be fully implemented in the initial version; requires proper configuration for security and backup operations

3.2 Database Software Utilities

Vendor	Product	Function
Oracle Corporation	MySQL Workbench	Used for modeling (EER diagrams)
Oracle Corporation	MySQL Server	Core database engine used for storing, managing, and retrieving data
GitHub	Git	Version control system used to track changes in database scripts and project files
GitHub	GitHub Platform	Online repository hosting for collaboration, backup, and project management

4 Architecture

4.1 Hardware Architecture

The hardware architecture of the Inventory Stock Management System is based on a simple client-server model designed to support efficient database operations and user access. The system operates on standard computing hardware, where the database is hosted on a machine running the MySQL server, which acts as the central data repository. This server is responsible for processing queries, managing transactions, and ensuring data integrity.

Users access the system through client machines such as desktop computers or laptops, which interact with the database either directly via tools like MySQL Workbench or through a future graphical user interface. The client devices send requests to the database server over a local network or standalone environment, and the server processes these requests and returns the appropriate results.

The hardware requirements for the system are modest, making it suitable for small to medium-scale deployment. A typical setup includes a computer with sufficient processing power, memory, and storage

capacity to handle database operations and store inventory data. As the system scales, the architecture can be upgraded to support dedicated servers, cloud-based infrastructure, or distributed systems to improve performance, availability, and reliability.

4.2 Software Architecture

The software architecture of the Inventory Stock Management System follows a layered design approach that separates concerns between data management and user interaction. At the core of the system is the database layer, implemented using MySQL, which is responsible for storing, managing, and retrieving all inventory-related data. This layer handles structured data operations such as queries, updates, and transaction processing using SQL.

Above the database layer is the application layer, which is intended to manage business logic and coordinate interactions between users and the database. Although the current implementation primarily focuses on the database, this layer can be extended in the future to include a graphical user interface or web-based system that allows users to perform operations such as adding products, recording transactions, and generating reports without directly interacting with SQL.

The architecture is designed to be modular and scalable, allowing additional components such as authentication systems, APIs, and reporting tools to be integrated without disrupting the existing database structure. This separation of layers improves maintainability, flexibility, and security, ensuring that the system can evolve to meet future requirements while maintaining efficient and reliable performance.

4.3 Datastores

The primary datastore for the Inventory Stock Management System is a relational database implemented using MySQL. This datastore is responsible for storing all structured data related to the system, including information about products, suppliers, customers, employees, roles, and transactions. Data is organized into multiple interconnected tables, with relationships enforced through primary and foreign key constraints to ensure integrity and consistency.

The database serves as the central repository where all data operations such as insertion, updates, deletion, and retrieval are performed using SQL queries. It supports efficient data access and management, enabling the system to handle inventory tracking, transaction processing, and reporting functionalities.

In addition to the main database, supplementary datastores may include external files such as SQL scripts used for schema creation, data population, and query execution. These files are typically stored in a version-controlled environment to support development, maintenance, and backup processes. As the system evolves, additional datastores such as cloud storage or integrated data services may be incorporated to enhance scalability, availability, and data sharing capabilities.

5 Database-Wide Design Decisions

The design of the Inventory Stock Management System database is guided by a set of decisions aimed at ensuring data integrity, consistency, scalability, and efficient performance. The database adopts a relational model in which data is organized into normalized tables to minimize redundancy and eliminate data anomalies. Entities such as products, suppliers, customers, employees, and transactions are separated into distinct tables, with relationships enforced through primary and foreign key constraints.

To maintain data accuracy and consistency, integrity constraints such as NOT NULL, UNIQUE, and referential integrity rules are applied across the schema. Indexing is implemented on key attributes to improve query performance, particularly for frequently accessed data such as product IDs and transaction records. The design also ensures that each transaction is properly linked to relevant entities, supporting traceability and accountability within the system.

The database is structured to support efficient CRUD (Create, Read, Update, Delete) operations and reporting functionalities through optimized SQL queries. Simplicity and clarity have been prioritized in the schema design to make the system easy to understand, maintain, and extend. Additionally, the design allows for future enhancements such as triggers for automatic stock updates, stored procedures for complex operations, and integration with external applications or user interfaces.

Overall, these design decisions ensure that the database remains reliable, maintainable, and adaptable to future system requirements while supporting the core functionality of inventory management.

5.1 Interfaces

The Inventory Stock Management System database is designed to interact with users and external components through well-defined input and output interfaces. The primary form of interaction is through SQL queries, which serve as inputs to the database for performing operations such as inserting new records, updating existing data, deleting entries, and retrieving information. These queries may be executed directly using tools like MySQL Workbench or indirectly through a future application layer or user interface.

The database accepts structured inputs in the form of valid SQL commands, ensuring that all data entered conforms to defined constraints such as data types, primary keys, and foreign key relationships. Input validation is enforced at the database level to maintain data integrity and prevent invalid or inconsistent entries.

In terms of outputs, the database produces query results in tabular format, which can be displayed to users, exported to external tools such as spreadsheets, or used to generate reports. These outputs include inventory status reports, sales summaries, supplier transaction histories, and other analytical data that support decision-making.

The system is also designed to support future integration with external systems such as web or desktop applications, reporting tools, and APIs. These interfaces will enable seamless data exchange between the database and other software components, allowing users to interact with the system through more intuitive graphical interfaces while maintaining the robustness of the underlying database structure.

5.2 Key Factors Influencing Design

The design of the Inventory Stock Management System database is influenced by both functional and non-functional requirements that ensure the system effectively supports inventory operations while maintaining reliability and efficiency. On the functional side, the database is required to support core operations such as storing and managing product details, tracking stock levels, recording supplier and customer information, and processing purchase and sales transactions. It must also support relationships between entities, enabling accurate linkage between employees, transactions, and inventory items, as well as providing the ability to generate meaningful reports for decision-making.

On the non-functional side, the design is guided by requirements such as data integrity, performance, scalability, and maintainability. Data integrity is ensured through the use of primary keys, foreign keys, and constraints that prevent inconsistencies and duplication. Performance considerations influence the use of indexing and optimized relational structures to ensure fast query execution even as data volume increases. Scalability is considered to allow the database to accommodate future growth in transactions and additional features without major restructuring. Maintainability is achieved through a clear and normalized schema design that makes it easier to update, extend, and debug the system. Additionally, security and reliability requirements influence the design by ensuring controlled access to data and minimizing risks of data loss or corruption.

Overall, these factors ensure that the database is both functionally complete and robust enough to support current and future operational needs.

5.3 Behavior

The database is designed to respond efficiently to user inputs and SQL queries by performing appropriate actions such as inserting, updating, retrieving, or deleting data while maintaining data integrity and consistency. Each valid request is processed according to defined relational rules, ensuring that relationships between tables (such as products, transactions, and employees) remain accurate. The system is optimized to provide fast response times for common queries through the use of indexing and normalized table structures, allowing most operations to execute in minimal time depending on data size and system resources. Standard SQL algorithms are used for data retrieval and joins to support reporting and analysis functions. In cases where invalid or unauthorized inputs are provided, the system rejects the request and returns appropriate error messages without affecting existing data, thereby ensuring stability and reliability of the database.

5.4 DBMS Platform

The database will be implemented using MySQL Community Server 8.0, selected for its reliability, performance, and wide support. This platform allows efficient data management, strong security features, and easy integration with web or desktop applications. Flexibility will be built into the design through normalization, modular table structures, and the use of scalable schemas that can be modified to accommodate future changes in requirements without major restructuring.

5.5 Security and Availability

The Inventory Stock Management System database implements security and access control measures to ensure data integrity, confidentiality, and reliable availability of information. Integrity controls are enforced through the use of primary keys, foreign keys, and relational constraints such as NOT NULL and UNIQUE, which prevent duplicate entries and maintain consistency across related tables such as Products, Transactions, and Employees. Access to database components is restricted based on user roles to ensure that only authorized users can perform specific operations on schemas, tables, and records.

Users are classified into different categories based on their responsibilities. Administrators have full access rights, including the ability to create, modify, and delete database structures and data. Employees have limited access, typically restricted to inserting and updating operational data such as transactions and product records, while being prevented from altering the database structure. In some cases, read-only users such as auditors or managers may be granted access to view reports and query data without making modifications.

To ensure availability, the database is designed to operate on a reliable MySQL server with regular backup procedures to prevent data loss in case of system failure. Access controls combined with backup strategies ensure that the system remains secure, consistent, and available for continuous use.

5.6 Backup and Restore Operations

The Inventory Stock Management System database implements backup and restore strategies to ensure data protection, recovery, and continuity in the event of system failure, corruption, or accidental data loss. The primary backup approach involves periodic full backups of the MySQL database using SQL dump files, which capture the entire database schema and data at specific intervals. These backups may be scheduled daily or weekly depending on system usage and stored securely on external storage devices or cloud-based repositories to prevent loss due to hardware failure.

During backup operations, the database is generally placed in a consistent state where write operations may be temporarily restricted to ensure data integrity, while read operations can still be allowed depending on system requirements. Restoration procedures involve importing the backup files back into the MySQL server to reconstruct the database to its previous state, ensuring minimal data loss and system downtime.

The system is primarily structured for standard relational data; therefore, it does not heavily involve non-standard data types such as video or sound. However, if such data were introduced in future enhancements, additional storage and backup considerations such as larger storage capacity and specialized handling mechanisms would be required. Overall, the backup and restore strategy ensures

reliability, disaster recovery capability, and continuous availability of the database system.

5.7 Maintenance

The maintenance of the Inventory Stock Management System database is designed to ensure long-term efficiency, consistency, and optimal performance. Regular maintenance activities include indexing key fields such as Product ID, Transaction ID, and Employee ID to improve query performance and enable faster data retrieval. Sorting and query optimization techniques are applied through structured SQL queries to enhance system responsiveness, especially when handling large datasets.

To maintain consistency and data integrity, synchronization between related tables is enforced using foreign key constraints, ensuring that updates in one table are accurately reflected across related records. Repacking and optimization of database tables are performed periodically to reclaim unused storage space and improve storage efficiency, especially after large delete or update operations. Automated disk management strategies provided by the MySQL engine are utilized to support efficient storage allocation and reduce fragmentation.

Database population is carried out through controlled insertion of validated data to ensure accuracy and avoid redundancy. In cases where legacy data is introduced, it is carefully cleaned, formatted, and migrated into the system to align with the current schema structure. Overall, these maintenance practices ensure that the database remains efficient, scalable, and reliable throughout its lifecycle.

6 Database Administrative Functions

Database Identification

Element	Element Name	Description
db_name	Database name	The name of the database created in MySQL, which is Inventory_stock_system.
db_path	Database path	The physical storage location of the MySQL database files managed internally by the MySQL server on the Linux system.
db_location	Database Location	The location of the database in relation to the application. The database is hosted on a local MySQL server running on Linux, accessed through the terminal interface for execution of SQL commands.
db_storage_path	Database Path	The full path of a location that is used by the database for placing automatic storage tables.`/home/howard/Desktop/Database/Inventory-Stock-Management-Database/DATABASE` — this is the project

		directory containing SQL scripts, schema files, and related database resources used for development and maintenance.

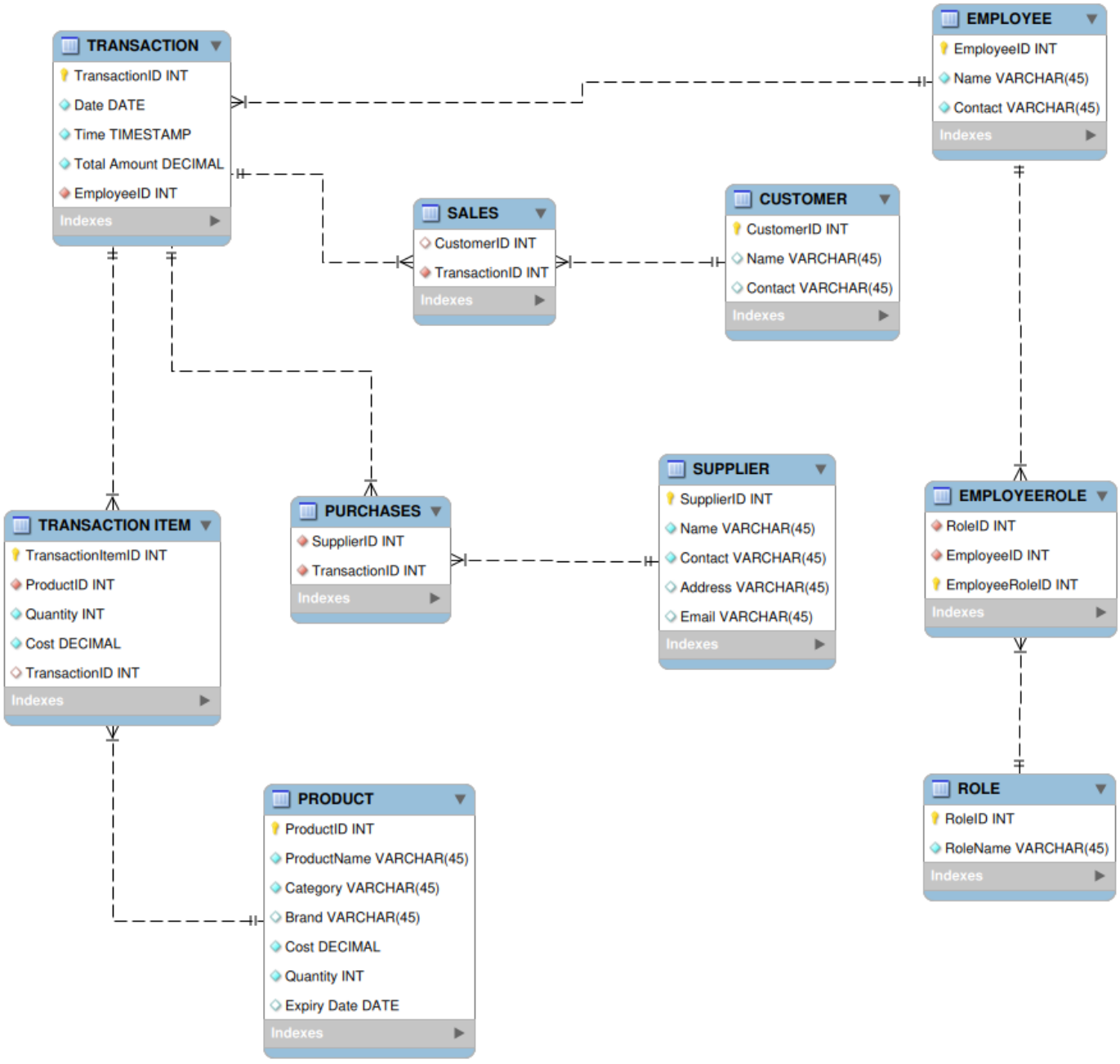
6.1 Schema Information

The overall structure of the database schema for the Inventory Stock Management System is based on a relational model designed to organize data into logically connected tables that represent real-world entities within a Supermarket. The schema is structured to ensure data normalization, reduce redundancy, and maintain consistency across all operations. It consists of core tables such as Product, Supplier, Customer, Employee, Role, Transaction, and TransactionItem, each representing a specific entity within the system. These tables are interconnected through defined relationships using primary and foreign keys, enabling efficient data linking and integrity enforcement.

Globally, the schema defines how data is stored, accessed, and related across the entire database system. Each table contains clearly defined attributes with specific data types, constraints, and validation rules to ensure accuracy and reliability of stored information. The schema also enforces referential integrity, ensuring that dependent records cannot exist without their associated parent records, such as transaction items being linked to valid transactions and products. Additionally, indexing is applied to key fields to improve query performance and support fast data retrieval. Overall, the schema provides a structured and scalable foundation that supports inventory tracking, transaction processing, and reporting functionalities within the system.

6.2 Physical Design

The diagram below illustrates the structure of the database showing entities, attributes, and relationships that are implemented physically in the MySQL database.



7 Detailed Database Design

7.1 Database Management System Files

The Inventory Stock Management System database is implemented using MySQL, and its physical implementation is organized into structured DBMS files that support database creation, data population, and query operations. The database files are logically separated into schema definition, data insertion, and query execution scripts to ensure modularity, maintainability, and ease of deployment.

The schema.sql file defines the physical structure of the database, including the creation of tables, primary keys, foreign keys, constraints, and relationships between entities such as Product, Supplier, Customer, Employee, Role, Transaction, and TransactionItem. This file represents the core schema definition and ensures that all tables are properly structured according to the relational model.

The data.sql file is used for populating the database with initial or sample data. It contains INSERT statements that insert records into the respective tables, allowing the database to be tested and validated with realistic data values. This supports system initialization and testing of relationships between entities.

The queries.sql file contains predefined SQL queries used for data retrieval, reporting, and analysis. These queries demonstrate system functionality such as retrieving stock levels, generating sales reports, and analyzing transaction records. This file also supports testing performance and validating database behavior under different query operations.

From a physical storage perspective, MySQL manages data internally using tables stored in database files with fixed page sizes handled by the InnoDB storage engine. Data is stored in structured pages that support efficient reading and writing operations. The system uses indexing on primary and foreign keys to support fast access methods, including direct lookups (random access) and optimized retrieval through indexed searches rather than full table scans.

To support performance objectives, indexes are applied on frequently accessed columns such as Product_ID, Transaction_ID, and Employee_ID. This improves query execution speed and reduces response time during reporting and transaction processing. Data access is primarily random access through indexed keys, while sequential access is used for full table scans during reporting operations.

The database does not currently implement physical partitioning; however, the design allows for future scalability through potential horizontal or vertical partitioning if data volume increases significantly. All data is currently stored in a centralized MySQL database on a Linux local server, ensuring simplicity and consistency. This compartmentalization into schema, data, and query files ensures separation of concerns, easier maintenance, and controlled deployment of the system.

A detailed Physical Data Model (PDM) ERD illustrating table structures and relationships is provided in the appendix for reference.rovide an appropriate level of detailed design of the DBMS files, based on the DBMS chosen. Describe support performance objectives. Include the following information, as appropriate (refer to the Data Dictionary):

.